CodePilot — AI Software Engineering Agent

Project Type: AI Engineering / Developer Tool
Goal: Build an autonomous AI agent capable of taking a software-development task, inspecting a codebase, making changes, running tests, diagnosing failures, and iterating until the task is completed or human intervention is required.

---

1. Project Overview

CodePilot is an AI-powered software engineering agent designed to work with real Git repositories.

A user provides:

1. A software repository
2. A natural-language development task

CodePilot autonomously:

Understand task
↓
Inspect repository
↓
Understand relevant code
↓
Create implementation plan
↓
Modify code
↓
Run tests / lint / build
↓
Analyze results
↓
Fix problems
↓
Repeat if necessary
↓

Generate final report

The system should behave more like an **AI software engineer** than a traditional chatbot.

The primary objective is not simply to generate code, but to demonstrate **agentic software engineering**: planning, tool usage, execution, verification, error recovery, and controlled autonomy.

---

2. Core Product Concept

A user should be able to provide a task such as:

> "Add pagination to the `/api/users` endpoint. Keep the existing API backwards compatible and add tests."

CodePilot should independently determine:

- where the relevant endpoint is
- how the existing architecture works
- which files need modification
- what implementation approach is appropriate
- what tests need to be added
- whether the implementation actually works

The user should not need to manually tell the agent:

> "Open `src/api/users.ts`."

The agent should discover that itself.

---

3. Target Users

Primary

- Software developers
- CS students
- Open-source contributors
- Development teams

Secondary

- Developers experimenting with AI agents
- Researchers studying coding agents
- Teams wanting automated maintenance/testing

---

4. Main Features

4.1 Repository Management

The system must support working with Git repositories.

Capabilities:

- Import local repository
- Clone Git repository
- Detect project type
- Detect programming language
- Detect package manager
- Detect test framework
- Detect build system
- Detect linting configuration
- Display repository structure

Supported repositories should initially focus on common projects such as:

- JavaScript
- TypeScript
- Python
- Java
- C++
- C#

Additional languages can be added later.

---

5. Task Input

The user provides a natural-language task.

Example:

Fix the bug where users can submit the login form
multiple times while the request is still processing.

The task interface should support:

- Task description
- Repository selection
- Optional branch selection
- Optional additional instructions
- Optional test command
- Optional build command

The user should be able to start an agent session with one action.

---

6. Repository Analysis

Before modifying anything, CodePilot should analyze the repository.

The agent should have access to tools for:

File inspection

- List files
- Read files
- Search files
- Search code
- Find symbols
- Inspect directory structure

Project understanding

The agent should identify:

- Entry points
- Important modules
- Dependencies
- Configuration
- Tests

- Build scripts
- Existing architecture

The agent should avoid unnecessarily reading the entire repository.

It should selectively inspect relevant files.

---

7. Planning System

Before making changes, the agent should produce an implementation plan.

Example:

Implementation Plan

1. Locate login form component.
2. Identify current submit-state handling.
3. Add request-in-progress state.
4. Disable submission while request is active.
5. Re-enable after request completion.
6. Add regression tests.
7. Run test suite.

The plan should be visible to the user.

The agent should then execute the plan step-by-step.

---

8. Agent Tool System

The AI should not directly have unrestricted access to the computer.

Instead, it interacts with the environment through defined tools.

File Tools

list_files
read_file
search_code
find_symbol

write_file
delete_file

Git Tools

git_status
git_diff
git_log
git_branch
create_branch
commit_changes

Execution Tools

run_command
run_tests
run_linter
run_build

Project Tools

detect_project
detect_test_framework
detect_package_manager

Agent Tools

create_plan
update_plan
request_approval
finish_task

The tool system should return structured results to the agent.

---

9. Code Modification

The agent should be capable of modifying existing code rather than simply generating new files.

Code changes should:

- Preserve existing architecture where practical
- Avoid unnecessary modifications
- Follow existing coding conventions
- Minimize unrelated changes

- Add tests when appropriate

Every modification must be recorded.

The UI should allow the user to inspect the resulting diff.

---

10. Execution & Testing

After making changes, CodePilot should verify them.

Depending on the project, it should be able to run:

Tests
Lint
Type checking
Build
Custom commands

Example:

Running tests...

42 tests
41 passed
1 failed

The failure should automatically be returned to the agent.

---

11. Autonomous Error Recovery

This is one of the most important features.

If tests fail, the agent should not immediately stop.

Example:

Implement change
↓
Run tests

↓
Tests fail
↓
Read failure output
↓
Inspect relevant code
↓
Determine cause
↓
Modify code
↓
Run tests again

The agent should have a configurable maximum number of iterations.

Example:

Maximum repair iterations: 5

If the agent cannot solve the problem after the limit:

TASK REQUIRES HUMAN INTERVENTION

The system should explain:

- What it attempted
- What failed
- What remains unresolved
- Relevant error messages
- Files changed

---

12. Human Approval System

Certain operations should require explicit user approval.

Examples:

- Deleting files
- Destructive shell commands
- Database modifications
- Installing dependencies
- Changing environment configuration

- Modifying sensitive configuration
- Executing potentially dangerous commands

Example:

⚠️ Approval Required

The agent wants to execute:

npm install some-package

Reason:
Required dependency for the requested implementation.

[Approve] [Reject]

The user should also be able to configure an autonomy level.

Modes

Safe

Most potentially impactful operations require approval.

Balanced

Normal development operations happen automatically, while dangerous operations require approval.

Autonomous

The agent operates with minimal interruption within the configured sandbox.

---

13. Sandboxed Execution

CodePilot should execute repository commands inside an isolated environment where practical.

The goal is to prevent repository code or agent-generated commands from having unrestricted access to the host system.

The sandbox should restrict:

- File system access

- Network access where possible
- System-level operations
- Credentials
- Sensitive environment variables

The system should clearly communicate that sandboxing is a security boundary and not assume that AI-generated commands are trustworthy.

---

14. Git Integration

Every task should ideally operate on a separate Git branch or isolated working copy.

Example:

```
main
 │
 └── codepilot/task-4821
```

The user should be able to see:

Files changed: 5
Insertions: +87
Deletions: -24

The system should provide:

- Git diff
- Changed files
- Commit history
- Branch information
- Optional commit creation
- Revert/reset functionality

The agent should never silently modify the user's primary branch.

---

15. Final Task Report

After completion, CodePilot should produce a structured report.

Example:

TASK COMPLETED

Task:
Add pagination to /api/users.

Status:
✓ Completed

Files changed:
5

Tests:
47 passed
0 failed

Build:
✓ Passed

Lint:
✓ Passed

Iterations:
2

Summary:
Implemented backwards-compatible pagination for
the users endpoint and added regression tests.

Changes:
• Added page/pageSize parameters
• Added pagination metadata
• Preserved existing API behavior
• Added 6 tests

---

16. Web Dashboard

The application should have a modern developer-oriented interface.

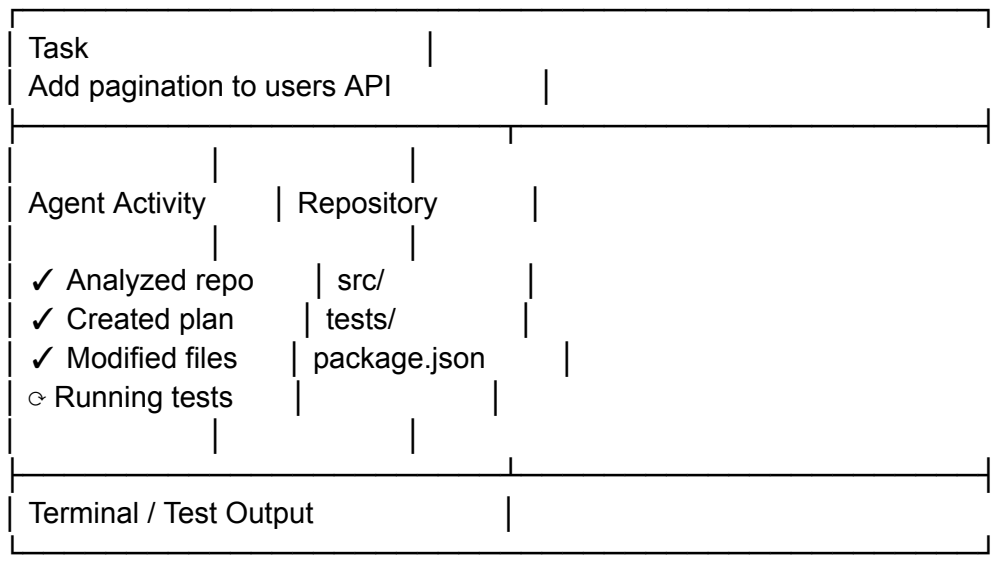
Dashboard

Display:

Projects
Recent Tasks
Agent Activity
Success Rate
Tasks Completed

Task Page

Main areas:



---

17. Real-Time Agent Activity

The user should be able to observe what the agent is doing.

For example:

15:42:01 Agent started

15:42:03 Inspecting repository

15:42:06 Found TypeScript project

15:42:08 Searching for users endpoint

15:42:11 Reading src/api/users.ts

15:42:18 Implementation plan created

15:42:23 Modifying src/api/users.ts

15:42:31 Adding tests

15:42:39 Running test suite

15:42:47 1 test failed

15:42:50 Analyzing failure

15:43:02 Applying fix

15:43:11 Running tests again

15:43:17 ✓ 47 tests passed

This is important because it makes the agent's behavior observable rather than presenting a black box.

---

18. Diff Viewer

The UI should contain a proper code-diff view.

Example:

- const users = await getUsers();

+ const page = Number(req.query.page ?? 1);
+ const pageSize = Number(req.query.pageSize ?? 20);
+
+ const users = await getUsers(page, pageSize);

Users should be able to inspect every modification.

---

19. Agent State

Each task should maintain persistent state.

Example:

```
Task
 ├── User request
 ├── Repository
 ├── Agent plan
 ├── Current step
 ├── Tool calls
 ├── Files modified
 ├── Test results
 ├── Errors
 ├── Iterations
 └── Final result
```

The agent should be able to recover from interruptions where possible.

---

20. Agent Memory / Context

The system should maintain relevant context throughout the task.

It should remember:

- Original task
- Current plan
- Completed steps
- Test results
- Previous errors
- Files inspected
- Files modified

However, it should avoid blindly accumulating the entire repository into the model context.

The system should prioritize relevant information.

---

21. MCP Support

CodePilot should support **Model Context Protocol (MCP)** as an extension mechanism.

The architecture should allow external tools to be connected to the agent.

Potential integrations:

```
CodePilot
  │
  ├── Filesystem
  ├── Git
  ├── GitHub
  ├── Database
  ├── Documentation
  └── Custom MCP servers
```

MCP should be treated as an optional tool layer rather than a hard dependency for the initial version.

---

22. Evaluation / Benchmarking

This is a major differentiating feature.

CodePilot should have a benchmark system containing predefined software-engineering tasks.

Example:

Task 001
Fix broken authentication

Task 002
Add API pagination

Task 003
Fix failing test

Task 004
Implement missing function

Task 005

Fix TypeScript compilation error

Each task should have:

- Repository
- Initial state
- Task description
- Expected behavior
- Tests
- Success criteria

The agent can then be evaluated objectively.

---

23. Metrics

The system should track:

Task success rate

Successful tasks / Total tasks

Test success

Tests passed / Tests executed

Repair iterations

Average attempts per task

Time

Average task completion time

Tool usage

Files read
Files modified
Commands executed
Tests executed

Failure categories

Planning failure
Implementation failure
Testing failure

Tool failure
Agent timeout
Human intervention

These metrics should be visible on an evaluation dashboard.

---

24. Agent Trace

Every agent action should be recorded.

Example:

Agent
├─ Thought/decision summary
├─ Tool call
├─ Tool result
├─ Decision
├─ Tool call
└─ ...

For privacy and usability, the system should expose **concise reasoning summaries and actions**, rather than relying on or displaying private chain-of-thought.

The trace should make the agent's behavior auditable.

---

25. Configuration

Users should be able to configure:

AI Model
Temperature / generation settings
Maximum iterations
Autonomy level
Test command
Build command
Lint command
Sandbox restrictions
MCP servers

Git behavior

---

26. AI Model Layer

The AI provider should be abstracted behind a model interface.

The system should not be tightly coupled to one model provider.

Conceptually:

```
AIModel
 ├── Local model
 ├── Provider A
 ├── Provider B
 └── Future providers
```

The initial implementation should prioritize **free/local inference**.

A local model runtime such as **Ollama** can be used so that the project does not require paid API access.

---

27. Required Technology / Tools

The exact implementation can change, but the project should use technologies in roughly these categories.

Frontend

- React
- Next.js
- TypeScript
- Tailwind CSS
- Monaco Editor or equivalent code editor
- Diff viewer

Backend

- Node.js
- TypeScript

- REST API and/or WebSockets/SSE

AI

- Ollama
- Local coding-capable LLM
- Tool/function calling
- Structured outputs

Agent

- Custom agent orchestration layer
- Optional agent framework
- MCP support

Development environment

- Git
- Docker
- Linux/WSL where appropriate

Storage

- SQLite
- ORM such as Prisma/Drizzle if desired

Testing

- Vitest/Jest for the application
- Project-specific test runners for repositories
- Automated benchmark runner

---

28. Security Requirements

Security should be considered a core feature rather than an afterthought.

The system must:

- Never expose host credentials to the model
- Avoid unrestricted shell access
- Isolate repository execution

- Require approval for dangerous operations
- Restrict network access where possible
- Prevent access outside the designated workspace
- Log executed commands
- Log file modifications
- Provide a way to terminate the agent
- Provide task timeouts
- Limit excessive tool calls

The agent should be treated as **untrusted software automation**.

---

29. Non-Goals

The initial project should **not** attempt to:

- Train its own LLM
- Replace professional IDEs
- Support every programming language
- Build a full enterprise deployment system
- Automatically deploy production applications
- Give the agent unrestricted computer access
- Build a general-purpose ChatGPT competitor

The focus is specifically:

> **Autonomous software engineering inside a controlled repository environment.**

---

30. Development Phases

Phase 1 — Core Agent

Repository
↓
Task
↓
AI
↓

Read files
↓
Modify files
↓
Run tests

Goal: prove the fundamental agent loop.

Phase 2 — Autonomous Repair

Add:

Test failure
↓
Analyze
↓
Modify
↓
Retest

Goal: agent can recover from mistakes.

Phase 3 — Git + Safety

Add:

- Git branches
- Diff viewer
- Approval system
- Sandbox
- Command restrictions

Goal: make the agent safe enough to demonstrate publicly.

Phase 4 — Web Interface

Add:

- Dashboard
- Task interface
- Live agent activity
- Terminal output
- Diff viewer
- Task history

Goal: make the project feel like an actual product.

Phase 5 — MCP

Add:

- MCP client
- External tools
- Tool management
- Custom MCP servers

Goal: demonstrate modern agent/tool architecture.

Phase 6 — Evaluation

Add:

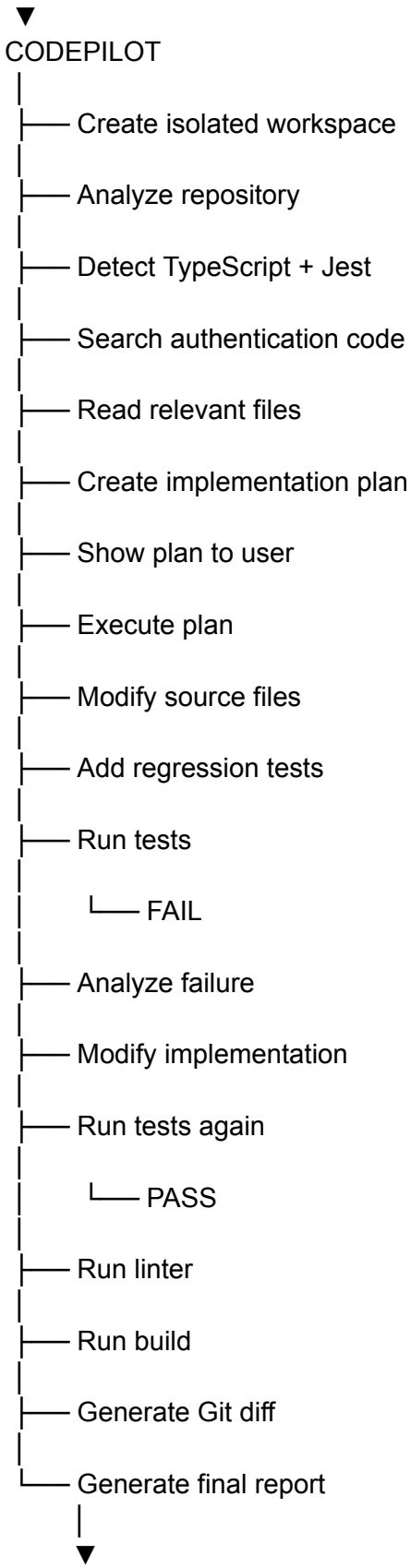
- Benchmark tasks
- Automated evaluation
- Success metrics
- Agent traces
- Performance dashboard

Goal: demonstrate that the system can be **measured**, not merely demonstrated.

---

31. Example End-to-End Workflow

USER
│
│ "Fix the authentication bug."


CODEPILOT
Create isolated workspace
Analyze repository
Detect TypeScript + Jest
Search authentication code
Read relevant files
Create implementation plan
Show plan to user
Execute plan
Modify source files
Add regression tests
Run tests
FAIL
Analyze failure
Modify implementation
Run tests again
PASS
Run linter
Run build
Generate Git diff
Generate final report

USER

---

32. Definition of Success

The project is considered successful when a user can give CodePilot a real repository and a reasonably well-defined software-development task and the system can independently:

1. Understand the repository
2. Identify relevant code
3. Produce a reasonable plan
4. Modify the code
5. Add or modify tests
6. Execute the project's verification commands
7. Detect failures
8. Attempt repairs
9. Produce a clean diff
10. Explain the completed work
11. Stop safely when human intervention is required

---

33. Resume Value

The project should demonstrate the following skills:

AI Engineering

- LLM integration
- Agent orchestration
- Tool calling
- Context management
- Structured outputs
- Agent evaluation

Software Engineering

- Git
- Testing
- Debugging
- Code analysis
- Backend architecture

- Frontend development

Modern AI Infrastructure

- MCP
- Local inference
- Sandboxing
- Agent observability
- Human-in-the-loop systems

System Design

- State management
- Asynchronous execution
- Security boundaries
- Fault tolerance
- Evaluation infrastructure

The central resume message should be:

> **Built an autonomous AI software-engineering agent capable of analyzing repositories, planning and implementing code changes, executing tests, recovering from failures, and producing verified Git diffs within a sandboxed environment.**

---

34. Final Product Vision

The finished project should feel less like:

> **"ChatGPT but for programmers."**

and more like:

> **"Give me a Git repository and a programming task. I'll investigate it, implement the change, test it, fix my mistakes, and show you exactly what I changed."**

That is the core identity of CodePilot.

Primary principle:

> **The AI is not the product. The autonomous software-engineering workflow is the product.**